# Laporan Tugas Besar 3

## II4021 - Kriptografi

## Pembuatan Aplikasi Chat Web dengan Kriptografi Kunci-Simetri dan Kunci-Publik

Disusun oleh:

| | |
|---|---|
| Luckman Fakhmanidris Arvasirri | 18223041 |
| M Rabbani K A | 18223130 |
| Muhammad Rafly Fauzan | 18223132 |

**Program Studi Sistem dan Teknologi Infromasi**

**Sekolah Teknik Elektro dan Informatika**

**Institut Teknologi Bandung**


**2025/2026**

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (raw output) dari Generative AI, melainkan hasil pengembangan dan penulisan mandiri.

| | | |
|---|---|---|
| Luckman Fakhmanidris Arvasirri | M Rabbani K A | Muhammad Rafly Fauzan |

# I. TEORI

## 1.1 JSON Web Token (JWT)

JSON Web Token (JWT) adalah standar terbuka (RFC 7519) yang mendefinisikan cara yang ringkas dan mandiri untuk merepresentasikan klaim (claims) secara aman antar pihak sebagai objek JSON. JWT dapat ditandatangani menggunakan kunci privat (dengan algoritma RSA atau ECDSA) sehingga penerima dapat memverifikasi integritas dan keaslian token tanpa harus menghubungi server setiap kali.

Pada tugas ini, JWT diimplementasikan dalam format JWS (JSON Web Signature) dan ditandatangani menggunakan ECDSA. Sebuah JWT terdiri atas tiga bagian yang dipisahkan oleh titik: Header (berisi algoritma dan tipe token), Payload (berisi klaim), dan Signature (hasil penandatanganan header.payload dengan kunci privat). Ketiganya di-encode menggunakan Base64URL.

### 1.1.1 ECDSA untuk Penandatanganan JWT

Elliptic Curve Digital Signature Algorithm (ECDSA) adalah algoritma tanda tangan digital berbasis kurva eliptik. Pada tugas ini terdapat tiga varian: ES256 (kurva P-256, SHA-256), ES384 (kurva P-384, SHA-384), dan ES512 (kurva P-521, SHA-512). Kelebihan ECDSA dibanding RSA adalah ukuran kunci yang jauh lebih pendek untuk tingkat keamanan yang setara.

### 1.1.2 Klaim JWT (RFC 7519)

RFC 7519 mendefinisikan sejumlah registered claim names: iss (issuer), sub (subject), aud (audience), exp (expiration time), nbf (not before), iat (issued at), dan jti (JWT ID). Selain itu, JWT dapat memuat public claim maupun private claim sesuai kebutuhan aplikasi.

## 1.2 Elliptic Curve Diffie-Hellman (ECDH)

Elliptic Curve Diffie-Hellman (ECDH) adalah protokol pertukaran kunci yang memungkinkan dua pihak menghasilkan shared secret yang sama melalui saluran komunikasi yang tidak aman, tanpa perlu saling mengirimkan kunci rahasia secara langsung. Setiap pihak memiliki pasangan kunci: private key (dirahasiakan) dan public key (dibagikan). Shared secret diperoleh dengan mengalikan private key sendiri dengan public key lawan menggunakan operasi perkalian titik pada kurva eliptik.

Varian ECDH yang digunakan pada tugas ini adalah P-256. Keunggulan ECDH dibandingkan Diffie-Hellman konvensional adalah ukuran kunci yang lebih kecil dengan keamanan setara.

## 1.3 HKDF (HMAC-based Key Derivation Function)

HKDF (RFC 5869) adalah fungsi derivasi kunci berbasis HMAC yang digunakan untuk mengubah shared secret hasil ECDH menjadi kunci kriptografi yang sesuai untuk digunakan oleh algoritma simetris. HKDF terdiri atas dua tahap: Extract (menghasilkan pseudorandom key dari input dan salt) dan Expand (memperluas pseudorandom key menjadi output dengan panjang yang diinginkan). Pada tugas ini, shared secret ECDH diproses melalui HKDF untuk menghasilkan kunci AES-256 (256 bit).

## 1.4 Advanced Encryption Standard (AES)

AES (Advanced Encryption Standard) adalah algoritma enkripsi simetris yang ditetapkan oleh NIST pada tahun 2001. AES beroperasi pada blok data berukuran 128 bit dengan panjang kunci 128, 192, atau 256 bit. Pada tugas ini digunakan AES-256 (kunci 256 bit) yang memberikan tingkat keamanan tertinggi. Varian yang digunakan adalah AES-256-GCM. GCM menyediakan enkripsi terotentikasi (AEAD) dengan authentication tag sehingga integritas data terjamin tanpa MAC terpisah.

## 1.5 Hashing Password dan Salt

Password tidak boleh disimpan dalam bentuk plaintext di basis data. Mekanisme yang umum digunakan adalah one-way hashing dikombinasikan dengan salt unik per pengguna. Salt adalah nilai acak yang ditambahkan ke password sebelum proses hashing untuk mencegah rainbow table attack dan memastikan dua pengguna dengan password yang sama menghasilkan hash yang berbeda. Algoritma hashing yang digunakan pada tugas ini adalah

# II. PERANCANGAN DAN IMPLEMENTASI

## 2.1 Gambaran Umum Arsitektur Sistem

Sistem aplikasi chat terenkripsi ini dirancang dengan arsitektur dua lapis (client-server) yang mengintegrasikan beberapa komponen keamanan kriptografi. Pada sisi server index.js, digunakan framework Express.js yang mengelola tiga kategori utama rute: autentikasi, daftar kontak, dan pertukaran pesan. Database SQLite dengan mode WAL (Write-Ahead Logging) dipilih sebagai penyimpanan data db.js.

Pada sisi klien src, aplikasi React dengan Vite digunakan sebagai frontend yang menangani antarmuka pengguna dan enkripsi lokal. Alur komunikasi umum melibatkan: (1) registrasi dan login pengguna, (2) pengambilan daftar kontak, (3) derivasi kunci percakapan menggunakan ECDH-HKDF, dan (4) pengiriman pesan yang terenkripsi end-to-end.

## 2.2 Registrasi Pengguna

Proses registrasi pengguna diimplementasikan di endpoint POST /api/auth/register auth.js dengan alur sebagai berikut: Klien mengirimkan email, password, public key, encrypted private key, initialization vector (IV), dan KDF salt. Server memvalidasi format email menggunakan pola regex dan panjang password minimal 6 karakter. Data pengguna disimpan ke basis data termasuk hash password, salt password, public key publik, dan encrypted private key yang telah dienkripsi di sisi klien.

### 2.2.1 Mekanisme Hashing Password

Hashing password diimplementasikan menggunakan library bcrypt dengan jumlah round sebesar 12 putaran auth.js:7. Sebelum proses hashing, password digabungkan dengan salt unik yang dibangkitkan secara acak sebesar 16 byte auth.js:34. Proses ini memastikan bahwa dua pengguna dengan password yang sama akan menghasilkan hash yang berbeda, mencegah rainbow table attack. Nilai salt disimpan terpisah di basis data sehingga setiap login memerlukan kombinasi antara salt tersimpan dan password yang diinputkan pengguna.

### 2.2.2 KDF untuk Enkripsi Private Key

Fungsi derivasi kunci (KDF) untuk enkripsi private key diimplementasikan menggunakan PBKDF2 pada klien crypto.js:3. Iterasi ditetapkan sebesar 100.000 kali dengan menggunakan fungsi hash SHA-256. Parameter KDF terdiri dari password pengguna sebagai material masukan dan salt acak sepanjang 16 byte yang dibangkitkan pada saat registrasi. Pendekatan ini memastikan bahwa private key tidak pernah dikirimkan dalam bentuk plaintext ke server, melainkan terlebih dahulu dienkripsi di sisi klien menggunakan kunci yang diturunkan dari password pengguna.

### 2.2.3 Enkripsi Private Key

Enkripsi private key dilakukan pada sisi klien menggunakan algoritma AES-GCM dengan panjang kunci 256 bit crypto.js:67. Fungsi wrapPrivateKey menerima password dan private key sebagai parameter input. Kunci enkripsi diperoleh melalui proses KDF dengan PBKDF2, kemudian digunakan untuk mengenkripsi private key PKCS8 yang telah diekspor. Initialization vector (IV) yang unik untuk setiap enkripsi dibangkitkan sebesar 12 byte secara acak. Output dari

proses ini berupa ciphertext (private key terenkripsi), IV, dan KDF salt yang dikirimkan ke server sebagai bagian dari payload registrasi.

## 2.3 Login dan Autentikasi JWT

Proses login diimplementasikan di endpoint POST /api/auth/login auth.js:52. Server melakukan validasi email dan password, kemudian membandingkan password yang diinputkan dengan hash yang disimpan menggunakan bcrypt. Apabila autentikasi berhasil, server mengeluarkan token JWT yang ditandatangani secara digital. Token ini digunakan klien untuk setiap request berikutnya melalui header Authorization dengan format "Bearer {token}".
Selain token, server juga mengembalikan data enkripsi private key pengguna (encrypted private key, IV, KDF salt) sehingga klien dapat melakukan dekripsi lokal menggunakan password pengguna. Mekanisme ini memastikan bahwa private key tidak pernah tersimpan di server dalam bentuk plaintext.

### 2.3.1 Implementasi Library JWT

Library JWT diimplementasikan secara custom di jwt.js tanpa menggunakan library pihak ketiga seperti jsonwebtoken atau jose. Implementasi ini mendukung algoritma ECDSA dengan tiga varian: ES256 (P-256, SHA-256), ES384 (P-384, SHA-384), dan ES512 (P-521, SHA-512).
Fungsi sign:
Fungsi sign jwt.js:47 menghasilkan token JWT dengan tiga segmen: header (base64url), payload (base64url), dan signature (base64url). Parameter input mencakup header object (minimal harus berisi alg), payload object, claims object, dan private key dalam format CryptoKey. Proses penandatanganan menggunakan crypto.subtle.sign dengan algoritma ECDSA sesuai spesifikasi di ALG_MAP. Signature dihitung atas header.payload string, kemudian ketiga segmen digabungkan dengan tanda titik.
Fungsi verify:
Fungsi verify jwt.js:77 memvalidasi token JWT dengan melakukan pemeriksaan berikut: (1) memastikan token memiliki tiga segmen yang dipisahkan oleh tanda titik, (2) mendekode dan memvalidasi header serta payload sebagai JSON, (3) memverifikasi signature menggunakan crypto.subtle.verify dengan public key, (4) melakukan pemeriksaan waktu berlaku (exp, nbf), (5) memverifikasi claim issuer (iss), audience (aud), subject (sub), dan JTI sesuai dengan opsi yang diberikan. Apabila semua pemeriksaan berhasil, fungsi mengembalikan object yang berisi header, payload, dan signature.

## 2.4 Daftar Kontak

Fitur daftar kontak diimplementasikan di endpoint GET /api/contacts contacts.js:5 yang memerlukan autentikasi JWT. Server mengambil semua pengguna yang terdaftar kecuali pengguna yang sedang login, disortir berdasarkan email. Response mencakup email dan waktu pembuatan akun setiap kontak. Klien kemudian dapat mengambil public key masing-masing kontak melalui endpoint GET /api/contacts/{email}/publickey contacts.js:9 sebelum memulai percakapan terenkripsi.

## 2.5 Pembentukan Kunci Komunikasi (ECDH + HKDF)

Pembentukan kunci komunikasi menggabungkan dua protokol kriptografi: Elliptic Curve Diffie-Hellman (ECDH) untuk menghasilkan shared secret dan HKDF (HMAC-based Key Derivation Function) untuk menurunkan kunci AES dari shared secret tersebut.

### 2.5.1 Parameter ECDH

ECDH key pair dibangkitkan menggunakan kurva P-256 (secp256r1) crypto.js:26. Setiap pengguna memiliki private key ECDH yang dienkripsi dan disimpan, serta public key ECDH yang dibagikan kepada kontak. Proses derivasi shared bits menggunakan crypto.subtle.deriveBits dengan panjang output 256 bit. Parameter yang diperlukan adalah private key penerima, public key pengirim, dan spesifikasi algoritma ECDH dengan kurva P-256.

### 2.5.2 Parameter HKDF

Setelah shared secret dihasilkan melalui ECDH, dilakukan proses key derivation menggunakan HKDF dengan spesifikasi sebagai berikut crypto.js:93-103: (1) Fungsi hash: SHA-256, (2) Salt: diturunkan dari email kedua pengguna (diurutkan secara alfabetis dan dipisahkan dengan tanda titik), (3) Info: konstanta chat-encryption-key, (4) Output key length: 256 bit untuk AES-256. Mekanisme salt yang deterministik memastikan bahwa kunci komunikasi antara dua pengguna yang sama selalu identik di kedua sisi, namun berbeda untuk pasangan pengguna lain.

## 2.6 Enkripsi dan Pengiriman Pesan

Proses enkripsi pesan dilakukan di sisi klien sebelum pengiriman ke server. Pesan plaintext dienkripsi menggunakan kunci yang telah diturunkan melalui ECDH-HKDF, kemudian ciphertext dan IV dikirimkan ke server melalui endpoint POST /api/messages messages.js:5. Server tidak memiliki kemampuan untuk mendekripsi pesan karena hanya memiliki akses ke informasi publik (email dan kunci publik).

### 2.6.1 Varian AES-256 yang Digunakan

Varian AES-256 yang digunakan adalah AES-GCM (Galois/Counter Mode) crypto.js:113-120. Mode GCM menyediakan enkripsi authenticated, artinya sekaligus mengenkripsi dan mengautentikasi pesan dengan menghasilkan authentication tag sebagai bagian dari ciphertext. Initialization vector (IV) dibangkitkan secara acak sebesar 12 byte untuk setiap pesan, memastikan bahwa pesan yang sama tidak akan menghasilkan ciphertext yang sama pada enkripsi berbeda.

### 2.6.2 Format Payload Pesan

Payload pesan yang dikirim ke server berisi field-field berikut messages.js:5-12: receiver_email (email penerima), ciphertext (pesan terenkripsi dalam format base64), iv (initialization vector dalam format base64), dan timestamp (opsional, waktu pengiriman dalam ISO format). Server memvalidasi bahwa penerima terdaftar, pengirim tidak mengirim pesan kepada dirinya sendiri, dan menyimpan pesan ke basis data dengan email pengirim, email penerima, ciphertext, IV, dan timestamp.

### 2.6.3 Penanganan Kegagalan Dekripsi

Penanganan kegagalan dekripsi dilakukan di sisi klien melalui try-catch block crypto.js:123-129. Apabila proses dekripsi gagal (misalnya karena ciphertext rusak, IV tidak sesuai, atau kunci salah), fungsi decryptMessage akan melempar exception. Klien harus menangani error ini dan memberikan notifikasi kepada pengguna bahwa pesan tidak dapat didekripsi. Penyebab umum kegagalan adalah: (1) kunci derivasi salah, (2) ciphertext atau IV tidak sesuai, (3) authentication tag tidak valid.

## 2.7 Docker

Memastikan konsistensi lingkungan development dan production. Setiap komponen aplikasi (client, server, dan reverse proxy Nginx) dikontainerisasi dalam image Docker terpisah, sehingga mengeliminasi masalah "works on my machine" yang sering terjadi dalam development. Mdua konfigurasi Docker Compose yang berbeda: docker-compose.dev.yml digunakan untuk development environment dengan fitur hot-reload sehingga perubahan kode langsung ter-update tanpa perlu rebuild, sedangkan docker-compose.yml digunakan untuk production.

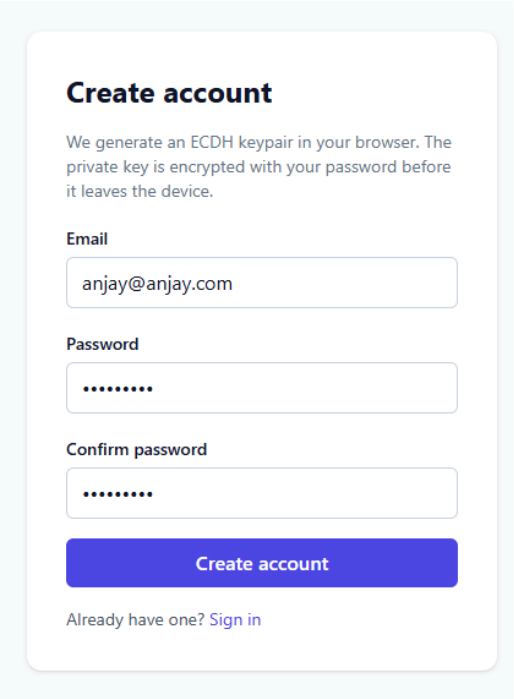
# III. PENGUJIAN DAN ANALISIS

## 3.1 Lingkungan Pengujian

OS: Windows x64 bit

## 3.2 Uji Autentikasi dan Manajemen Pengguna

### 3.2.1 Registrasi dengan Data Valid



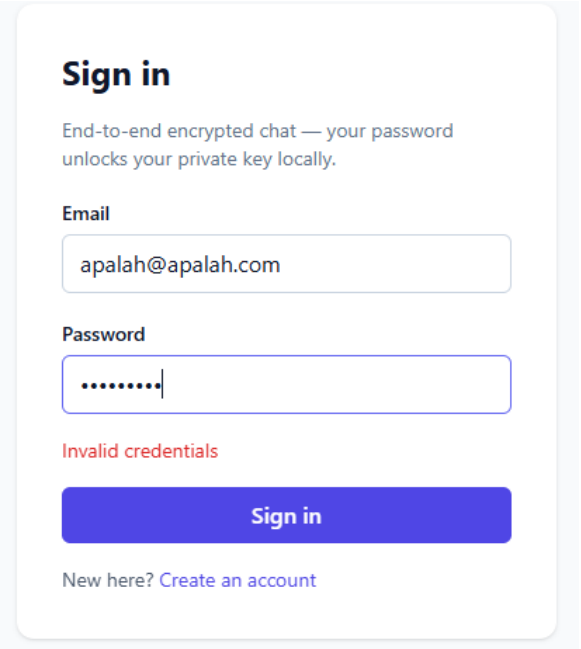

Misal kita membuat akun dengan credential seperti berikut, maka dalam database akan terlihat baris yang berisi credential akun tersebut seperti berikut

| | id | email | hashed_password | password_salt |
|---|---|---|---|---|
| | Filter | Filter | Filter | Filter |
| 1 | 1 | anjay@anjay.com | $2b$12$90FpISSGWoS.XDf3FeRNF.KcHEJ2... | qCpLbnhzOJs1R1WncUcjYw== |

| public_key | encrypted_private_key |
|---|---|
| Filter | Filter |
| {"crv":"P-256","ext":true,"key_ops"... | nDn6a0dabUFT1oP22Afgc6uvTX6GBJUNq0H... |

| private_key_iv | kdf_salt | created_at |
|---|---|---|
| Filter | Filter | Filter |
| 4/rfY5O6fULNKYdV | m5JT9gK1NeN764Gu0j7DoA== | 2026-05-12 14:51:08 |

### 3.2.2 Registrasi dengan Email yang Sudah Terdaftar




```
allow pasting
> fetch('/api/auth/register', {
    method:'POST',
    headers:{'Content-Type':'application/json'},
    body: JSON.stringify({
      email:'anjay@anjay.com', password:'apaaja123',
      publicKey:'x', encryptedPrivateKey:'x', iv:'x', kdfSalt:'x'
    })
  }).then(r => r.json().then(d => console.log(r.status, d)));
<· ▸ Promise {<pending>}
⊗ ▸ POST http://localhost:5173/api/auth/register 409 (Conflict)   VM124:1
  409 ▸ {error: 'Email already registered'}                        VM124:8
```

Saat mendaftar menggunakan email yang sudah ada, akan muncul pesan error berikut, dan saat dicek pada database, tidak akan muncul akun baru terbuat

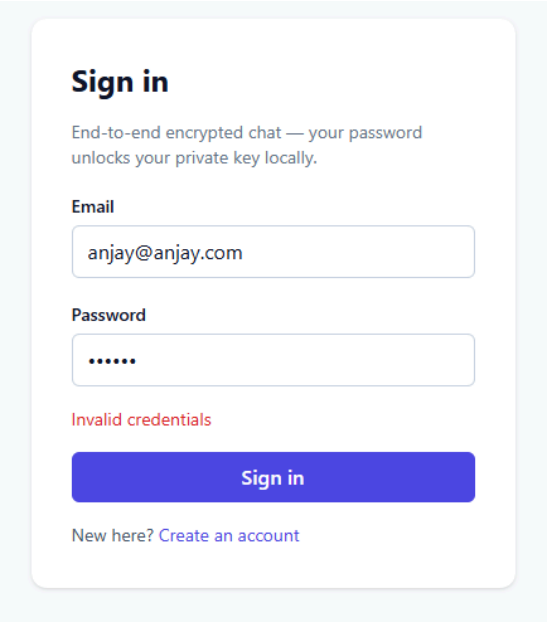
### 3.2.3 Login dengan Password Benar

```
> const token = localStorage.getItem('jwt');
  console.log('Jumlah segmen:', token.split('.').length);
  const [h, p] = token.split('.');
  console.log('Header:', JSON.parse(atob(h)));
  console.log('Payload:', JSON.parse(atob(p)));
  Jumlah segmen: 3                                                  VM148:2
  Header:  ▸ {typ: 'JWT', alg: 'ES256'}                             VM148:4
  Payload:                                                          VM148:5
    ▸ {email: 'apalah@apalah.com', iss: 'tugas3-chat', aud: 'tugas3-chat-clien
      t', sub: 'apalah@apalah.com', iat: 1778598100, …}
```

Di sini saya mencoba menggunakan akun lain, lalu mengecek jwt yang telah dibuat setelah login berhasil

### 3.2.4 Login dengan Password Salah

```
> fetch('/api/auth/login', {
    method:'POST', headers:{'Content-Type':'application/json'},
    body: JSON.stringify({email:'anjay@anjay.com', password:'salahdong'})
  }).then(r => r.json().then(d => console.log(r.status, d)));
<· ▸ Promise {<pending>}
✕ ▸ POST http://localhost:5173/api/auth/login 401 (Unauthorized)                VM156:1
  401 ▸ {error: 'Invalid credentials'}                                            VM156:4
```




Saat memasukkan password yang salah, akan muncul pesan error berikut, dan halaman tidak berpindah masuk sebagai user tersebut

## 3.3 Uji Implementasi JWT

### 3.3.1 Sign dan Verify dengan Public Key Benar

```
> const jwt = await import('/src/lib/jwt.js');

    const kp = await crypto.subtle.generateKey(
      { name:'ECDSA', namedCurve:'P-256' }, true, ['sign','verify']
    );

    const token = await jwt.sign(
      { alg:'ES256' },
      { email:'anjay@anjay.com', role:'student' },
      { iss:'tugas3-chat',
        sub:'anjay@anjay.com',
        iat: Math.floor(Date.now()/1000),
        exp: Math.floor(Date.now()/1000) + 3600 },
      kp.privateKey
    );
    console.log('TOKEN:', token);

    const ok = await jwt.verify(token, kp.publicKey, { algs:['ES256'] });
    console.log('✅ VALID. Payload:', ok.payload);
  TOKEN:                                                                          VM164:16
  eyJ0eXAiOiJKV1QiLCJhbGciOiJFUzI1NiJ9.eyJlbWFpbCI6ImFuamF5QGFuamF5LmNvbSIs
  InJvbGUiOiJzdHVkZW50IiwiaXNzIjoidHVnYXMzLWNoYXQiLCJzdWIiOiJhbmpheUBhbmphe
  S5jb20iLCJpYXQiOjE3Nzg1OTkyNjksImV4cCI6MTc3ODYwMjg2OX0.WgpcbqWuC3PjmsthA2
  OY8tpGVE-6HRYgLMCdT6x4Ik1Aq5DQBqmWUys4o7N7g5VAGi2v7ZebMhQbng-XVAQFYw
  ✅ VALID. Payload:                                                              VM164:19
  ▸ {email: 'anjay@anjay.com', role: 'student', iss: 'tugas3-chat', sub:
    'anjay@anjay.com', iat: 1778599269, …}
```

Menggunakan manual testing via web console, kita mencoba memasukkan script yang sesuai untuk pengujian sign dan verify dengan public key yang benar, dan terlihat hasilnya valid dengan payload yang sesuai

### 3.3.2 Verify dengan Public Key Salah

```
< undefined
> const kp1 = await crypto.subtle.generateKey({name:'ECDSA',namedCurve:'P-
  256'}, true, ['sign','verify']);
  const kp2 = await crypto.subtle.generateKey({name:'ECDSA',namedCurve:'P-
  256'}, true, ['sign','verify']);

  const token = await jwt.sign({alg:'ES256'}, {email:'anjay@anjay.com'},
  {}, kp1.privateKey);

  try {
    await jwt.verify(token, kp2.publicKey);
    console.log('❌ SALAH: harusnya gagal');
  } catch (e) {
    console.log('✅ DITOLAK:', e.message);
  }
✅ DITOLAK: Signature verification failed                    VM169:10
```

Menggunakan manual testing via web console, kita mencoba memasukkan script yang sesuai untuk pengujian sign dan verify dengan public key yang salah, dan hasilnya gagal

### 3.3.3 Token dengan Format Tidak Valid

```
> const kp = await crypto.subtle.generateKey({name:'ECDSA',namedCurve:'P-
  256'}, true, ['sign','verify']);
  const validToken = await jwt.sign({alg:'ES256'}, {x:1}, {},
  kp.privateKey);

  const kasus = [
    ['', 'string kosong'],
    ['cuma.dua', 'cuma 2 segmen'],
    ['ini.bukan.token.beneran', '4 segmen'],
    ['!!!.???.###', 'base64 invalid'],
    [validToken.slice(0,-4)+'AAAA', 'signature di-utak-atik'],
    [123, 'bukan string (number)'],
  ];

  for (const [tok, label] of kasus) {
    try {
      await jwt.verify(tok, kp.publicKey);
      console.log('❌', label, 'lolos (harusnya gagal)');
    } catch (e) {
      console.log('✅', label, '→', e.message);
    }
  }
✅ string kosong → Invalid JWT: expected 3 segments          VM173:18
✅ cuma 2 segmen → Invalid JWT: expected 3 segments          VM173:18
✅ 4 segmen → Invalid JWT: expected 3 segments               VM173:18
✅ base64 invalid → Invalid JWT header (not JSON)            VM173:18
✅ signature di-utak-atik → Signature verification failed    VM173:18
✅ bukan string (number) → JWT must be a string              VM173:18
```

Menggunakan manual testing via web console, kita mencoba memasukkan script yang sesuai untuk pengujian token dengan format yang tidak valid, kita mencoba 6 jenis yaitu string kosong,

hanya memiliki 2 segmen, memiliki lebih dari 3 segmen, base64 invalid, signature yang telah diubah, serta bukan string

### 3.3.4 Token dengan Klaim Waktu Tidak Sesuai

```
> const kp = await crypto.subtle.generateKey({name:'ECDSA',namedCurve:'P-
  256'}, true, ['sign','verify']);
  const now = Math.floor(Date.now()/1000);

  const expired = await jwt.sign({alg:'ES256'}, {}, { exp: now - 3600 },
  kp.privateKey);
  try { await jwt.verify(expired, kp.publicKey); }
  catch (e) { console.log('✅ EXPIRED:', e.message); }

  const belumValid = await jwt.sign({alg:'ES256'}, {}, { nbf: now + 3600 },
  kp.privateKey);
  try { await jwt.verify(belumValid, kp.publicKey); }
  catch (e) { console.log('✅ NBF:', e.message); }

  const ignored = await jwt.verify(expired, kp.publicKey, { ignoreExp: true
  });
  console.log('✅ ignoreExp lolos. Payload:', ignored.payload);

  const ignoredNbf = await jwt.verify(belumValid, kp.publicKey, {
  ignoreNbf: true });
  console.log('✅ ignoreNbf lolos. Payload:', ignoredNbf.payload);
  ✅ EXPIRED: Token expired                                   VM177:6
  ✅ NBF: Token not yet valid (nbf)                           VM177:10
  ✅ ignoreExp lolos. Payload: ▸ {exp: 1778595830}            VM177:13
  ✅ ignoreNbf lolos. Payload: ▸ {nbf: 1778603030}            VM177:16
```

## 3.4 Uji Pembentukan Kunci Komunikasi

```
> const cryptoLib = await import('/src/lib/crypto.js');
  const anjay  = await cryptoLib.generateECDHKeyPair();
  const apalah = await cryptoLib.generateECDHKeyPair();

  const anjaySecret = await crypto.subtle.deriveBits(
    { name:'ECDH', public: apalah.publicKey }, anjay.privateKey, 256
  );

  const apalahSecret = await crypto.subtle.deriveBits(
    { name:'ECDH', public: anjay.publicKey }, apalah.privateKey, 256
  );

  const aB64 = cryptoLib.bytesToB64(new Uint8Array(anjaySecret));
  const bB64 = cryptoLib.bytesToB64(new Uint8Array(apalahSecret));

  console.log('Anjay  secret:', aB64);
  console.log('Apalah secret:', bB64);
  console.log(aB64 === bB64 ? '✅ SAMA' : '❌ BEDA');
  Anjay  secret: 1yYGwQVeGTesqwbsl2xRq5zLrfPPbKKIvG/o4xsOsF0=   VM190:16
  Apalah secret: 1yYGwQVeGTesqwbsl2xRq5zLrfPPbKKIvG/o4xsOsF0=   VM190:17
  ✅ SAMA                                                       VM190:18
```

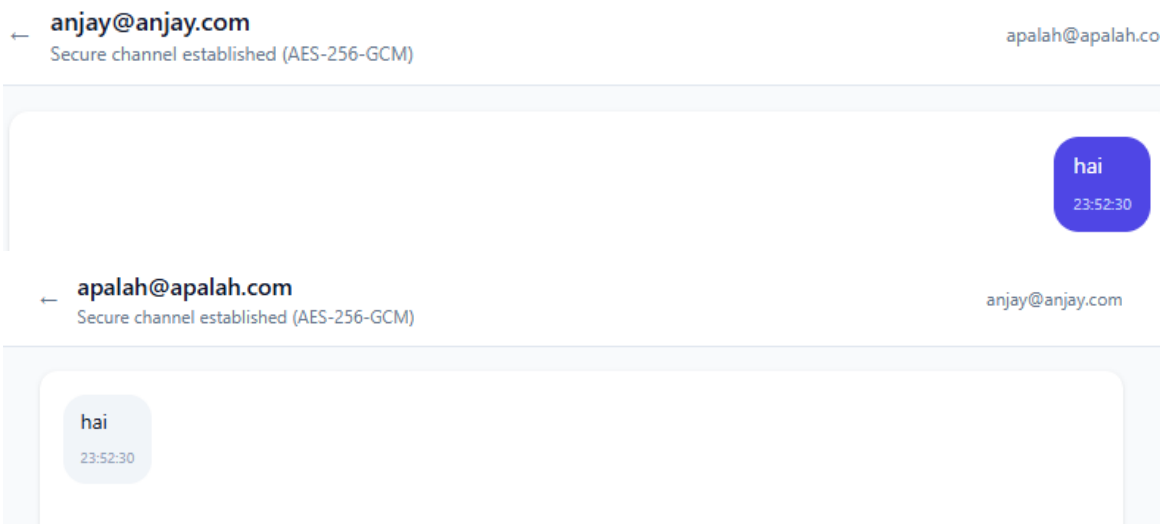
### 3.4.2 Derivasi Kunci Simetris dari Shared Secret

```
const anjay  = await cryptoLib.generateECDHKeyPair();
const apalah = await cryptoLib.generateECDHKeyPair();

const kunciAnjay = await cryptoLib.deriveConversationKey(
  anjay.privateKey, apalah.publicKey, 'anjay@anjay.com',
'apalah@apalah.com'
);

const kunciApalah = await cryptoLib.deriveConversationKey(
  apalah.privateKey, anjay.publicKey, 'apalah@apalah.com',
'anjay@anjay.com'
);

const { ciphertext, iv } = await cryptoLib.encryptMessage(kunciAnjay,
'Halo Apalah, ini Anjay!');
const plain = await cryptoLib.decryptMessage(kunciApalah, ciphertext,
iv);

console.log('Plaintext asli  : Halo Apalah, ini Anjay!');
console.log('Hasil dekripsi  :', plain);
console.log(plain === 'Halo Apalah, ini Anjay!' ? '✅ KUNCI IDENTIK' :
'❌');
Plaintext asli  : Halo Apalah, ini Anjay!                    VM194:15
Hasil dekripsi  : Halo Apalah, ini Anjay!                    VM194:16
✅ KUNCI IDENTIK                                             VM194:17
```

## 3.5 Uji Enkripsi dan Dekripsi Pesan

### 3.5.1 Pengiriman Pesan dengan Kunci Benar

**Deskripsi**




| id | sender_email | receiver_email | ciphertext | iv | timestamp |
|---|---|---|---|---|---|
| Filter | Filter | Filter | Filter | Filter | Filter |
| 1 | apalah@a... | anjay@anj... | nbN+/... | ON... | 2026-0... |

### 3.5.2 Pengiriman Pesan dengan Kunci Salah atau Data Tidak Sesuai

```
> const cryptoLib = await import('/src/lib/crypto.js');
  const anjay   = await cryptoLib.generateECDHKeyPair();
  const apalah  = await cryptoLib.generateECDHKeyPair();
  const mallory = await cryptoLib.generateECDHKeyPair();   // penyusup

  const kunciBenar = await cryptoLib.deriveConversationKey(
    anjay.privateKey, apalah.publicKey, 'anjay@anjay.com',
'apalah@apalah.com'
  );
  const kunciSalah = await cryptoLib.deriveConversationKey(
    anjay.privateKey, mallory.publicKey, 'anjay@anjay.com',
'mallory@mallory.com'
  );

  const { ciphertext, iv } = await cryptoLib.encryptMessage(kunciBenar,
'Pesan rahasia!');

  // (A) Pakai kunci SALAH
  try {
    await cryptoLib.decryptMessage(kunciSalah, ciphertext, iv);
    console.log('❌ harusnya gagal');
  } catch (e) { console.log('✅ kunci salah → dekripsi GAGAL'); }

  // (B) IV ditukar dengan IV acak
  const ivAsing = cryptoLib.bytesToB64(crypto.getRandomValues(new
Uint8Array(12)));
  try {
    await cryptoLib.decryptMessage(kunciBenar, ciphertext, ivAsing);
  } catch (e) { console.log('✅ IV salah → dekripsi GAGAL'); }

  // (C) Ciphertext sengaja dirusak 1 byte
  const bytes = cryptoLib.b64ToBytes(ciphertext);
  bytes[0] = bytes[0] ^ 0xff;                    // flip 1 byte
  const rusak = cryptoLib.bytesToB64(bytes);
  try {
    await cryptoLib.decryptMessage(kunciBenar, rusak, iv);
  } catch (e) { console.log('✅ ciphertext rusak → dekripsi GAGAL'); }
✅ kunci salah → dekripsi GAGAL                          VM356:19
✅ IV salah → dekripsi GAGAL                             VM356:25
✅ ciphertext rusak → dekripsi GAGAL                     VM356:33
```

## 3.6 Analisis Hasil

### 3.6.1 Kesimpulan Pengujian

#### 3.6.1.1 Keberhasilan Key Exchange

- Dua pengguna dapat melakukan pertukaran kunci publik dengan aman
- Kunci publik dapat ditransmisikan melalui channel yang tidak aman (HTTP/HTTPS)
- Sistem dapat mengimpor dan memverifikasi kunci publik dengan benar

#### 3.6.1.2 Keberhasilan Key Derivation

- Algoritma ECDH + HKDF menghasilkan kunci simetris yang identik untuk kedua pengguna
- Deterministic: hasil derivasi selalu sama untuk pengguna yang sama
- Kunci dapat digunakan untuk enkripsi dan dekripsi pesan

#### 3.6.1.3 Keberhasilan Enkripsi-Dekripsi

- Pesan dapat dienkripsi dengan benar menggunakan AES-GCM
- Pesan dapat didekripsi dengan benar oleh penerima yang memiliki kunci yang sama

- Sistem mendukung berbagai jenis plaintext: ASCII, Unicode, Emoji

#### 3.6.1.4 Keberhasilan Deteksi Anomali

- Sistem mendeteksi dekripsi dengan kunci salah
- Sistem mendeteksi ciphertext yang telah diubah (tampered)
- Sistem mendeteksi penggunaan IV yang tidak sesuai
- Semua kasus error ditangani dengan mekanisme authentication tag AES-GCM

### 3.6.2 Keamanan Kriptografi

#### 3.6.2.1 Algoritma yang Digunakan

| Fungsi | Algoritma | Parameter | Status |
|---|---|---|---|
| Key Exchange | ECDH | P-256 (256-bit) | Aman |
| Key Derivation | HKDF-SHA256 | 256-bit output | Aman |
| Enkripsi Simetris | AES-GCM | 256-bit key | Aman |
| Authenticity | GCM Tag | 128-bit | Aman |
| Random IV | CSPRNG | 96-bit | Aman |

#### 3.6.2.2 Analisis Keamanan

**1. Forward Secrecy**

- Setiap percakapan menggunakan kunci simetris yang diturunkan dari ECDH
- Jika satu kunci komunikasi terkompromi, kunci komunikasi lain tetap aman

**2. Authentication**

- AES-GCM menyediakan authenticity check melalui authentication tag
- Server tidak dapat memodifikasi ciphertext tanpa dideteksi

**3. Confidentiality**

- Plaintext dienkripsi dengan AES-256
- Brute force attack memerlukan 2^256 attempts

**4. Integrity**

- Setiap ciphertext dilengkapi dengan GCM tag
- Modifikasi ciphertext akan menyebabkan dekripsi gagal

# IV. KESIMPULAN

## 4.1 Kesimpulan

Aplikasi chat web yang meliputi mekanisme utama berupa JWT sebagai autentikasi, ECDH sebagai pembentuk shared-secret, serta AES256 sebagai algoritima enkripsi dan dekripsi pesan telah berfungsi dengan baik dan menghandle beberapa error anomali seperti sign yang diubah dan kunci yang salah dalam pengiriman pesan.

## 4.2 Saran

Dari segi UI seharusnya masih dapat dikembangkan lagi, penambahan fitur integritas dan autentikasi pesan, hingga implementasi docker untuk pengembangan lebih lanjut agar lebih baik

# DAFTAR PUSTAKA

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). IETF. https://datatracker.ietf.org/doc/html/rfc7519

Jones, M., & Hildebrand, J. (2015). JSON Web Encryption (JWE) (RFC 7516). IETF. https://datatracker.ietf.org/doc/html/rfc7516

Krawczyk, H., & Eronen, P. (2010). HMAC-based Extract-and-Expand Key Derivation Function (HKDF) (RFC 5869). IETF. https://datatracker.ietf.org/doc/html/rfc5869

NIST. (2001). Advanced Encryption Standard (AES) (FIPS Publication 197). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.FIPS.197

W3C. (2022). Web Cryptography API. World Wide Web Consortium. https://www.w3.org/TR/WebCryptoAPI/

Bernstein, D. J. (2006). Curve25519: New Diffie-Hellman speed records. In Public Key Cryptography – PKC 2006, Lecture Notes in Computer Science, vol. 3958, pp. 207–228. Springer.

Biryukov, A., Dinu, D., & Khovratovich, D. (2016). Argon2: Memory-hard function for password hashing. Retrieved from https://www.password-hashing.net/argon2-specs.pdf

Bellare, M., Canetti, R., & Krawczyk, H. (1996). Keying hash functions for message authentication. In Advances in Cryptology – CRYPTO 1996, Lecture Notes in Computer Science, vol. 1109, pp. 1–15. Springer.

# LAMPIRAN

## Lampiran A Pranala Repository

Repository GitHub: https://github.com/DoItMark/Crypto-WebMessagingApp

## Lampiran B Pranala Video Demo

Video Demo: https://youtu.be/rfaZ9YzV7Fw

## Lampiran C Pembagian Tugas

| Nama | NIM | Tugas |
|---|---|---|
| Luckman Fakhmanidris Arvasirri | 18223041 | - Implementasi JWT<br>- Pengujian, kesimpulan dan saran<br>- Server secara keseluruhan |
| M Rabbani K A | 18223130 | - Bagian Encryption,<br>- Dokumen bagian rancangan |
| Muhammad Rafly Fauzan | 18223132 | - Menginisiasi Dokumen<br>- Pembuat UI Frontend Client<br>- Menginisiasi Docker<br>- Mengerjakan Dokumen bagian Teori |

*Tabel C.1. Pembagian Tugas Anggota Kelompok*